1º DAW Miguel Ángel Merino Campos

Ejercicios Tema 5. Diseño y realización de pruebas con JUnit

Consideraciones En todos los ejercicios debes entregar:

1. Análisis del método o programa

- **Qué hace.**
- **Qué entradas admite.**
- **Qué salidas produce.**
- **Casos normales y casos límite.**

2. Complejidad ciclomática

- **Cálculo razonado del número mínimo de tests para cubrir las decisiones.**

3. Diseño de casos de prueba Para cada caso:

- **ID • Descripción**
- **Entrada**
- **Resultado esperado**
- **Oráculo**
- **Rama o camino que cubre**

4. Implementación en JUnit

- **Clase ...Test en src/test/java**
- **Métodos de test bien nombrados**
- **Uso de assertEquals, assertTrue, assertThrows, assertDoesNotThrow cuando proceda.**

Ejercicio 1. Método signo Se proporciona el siguiente método

```
 public static int signo(int x) {
 if (x > 0) { return 1;
 } else if (x < 0) { return -1;
} return 0; }
```

Tareas

1. Análisis del método o programa

- **Qué hace.**
  Nos indica el signo del número que usemos como dato si es positivo , negativo, o 0.
- **Qué entradas admite.**
  admite números enteros
- **Qué salidas produce.**
  1,0,-1 depende del dato
- **Casos normales y casos límite.**
  Casos normales:
  Número positivo – 1
  Número negativo – -1
  Casos límite:
  El 0 – 0

2. Calcula la complejidad ciclomática.

M=2+1.

Hay que hacer mínimo 3 tests.

3. Diseña el conjunto mínimo de pruebas para cubrir todas las ramas.

ID	Descripción	Entrada	R.Esperado	Oráculo	Camino
TC01	Número positivo	4	1	signo(5) == 1	if (x > 0)
TC02	Número negativo	-2	-1	signo(-2) == -1	else if (x < 0)
TC03	Valor cero (límite)	0	0	signo(0) == 0	no llegamos entrar en if

4. Implementa la clase SignoTest con JUnit.

implementada la clase SignoTest en JUnit, he hecho una clase main llamada metodoSigno, tiene el método abajo y en el main he hecho una pequeña prueba que pide el numero y te da el resultado del método aparte de la clase metodoSignoTest en el apartado Test.

Ejercicio 2. **Método clasificarEdad Se proporciona el siguiente método:**

```
public static String clasificarEdad(int edad) {
 if (edad < 0) {
throw new IllegalArgumentException("Edad no válida");
} if (edad < 6) {
return "Infancia";
} else if (edad < 12) {
 return "Niñez";
} else if (edad < 18) { return "Adolescencia";
} else if (edad < 25) {
 return "Juventud";
} else if (edad < 60) {
 return "Adultez";
 } else {
return "Vejez";
}
}
```

Tareas

1. Análisis del método o programa

• Qué hace.

Te devuelve según la edad que pongas en qué etapa de la vida esta eda edad

• Qué entradas admite.

Admite números enteros

• Qué salidas produce.

"Infancia"
"Niñez"
"Adolescencia"
"Juventud"
"Adultez"
"Vejez"
Excepción si la edad < 0

• **Casos normales y casos límite.**

Casos normales

3 – Infancia
10 – Niñez
15 – Adolescencia
20 – Juventud
40 – Adultez
70 – Vejez

Casos límite

-1 – excepción
0 – Infancia
5 – Infancia
6 – Niñez
11 – Niñez
12 – Adolescencia
17 – Adolescencia
18 – Juventud
24 – Juventud
25 – Adultez
59 – Adultez
60 – Vejez

2. Calcula la complejidad ciclomática.

edad < 0
edad < 6
edad < 12
edad < 18
edad < 25
edad < 60

-Total decisiones = 6

M = 6 + 1 = 7

Hay que hacer mínimo 7 tests.

3. Diseña el conjunto mínimo de pruebas para cubrir todas las ramas.

ID	Descripción	Entrada	Esperado	Oráculo	Camino
TC01	Edad negativa	-1	Excepción	lanza IllegalArgumentException	edad < 0
TC02	Infancia	4	"Infancia"	clasificarEdad(5) == "Infancia"	edad < 6
TC03	Niñez	9	"Niñez"	clasificarEdad(6) == "Niñez"	edad < 12
TC04	Adolescenci	16	"Adolescencia"	clasificarEdad(12) == "Adolescencia"	edad < 18
TC05	Juventud	21	"Juventud"	clasificarEdad(18) == "Juventud"	edad < 25
TC06	Adultez	32	"Adultez"	clasificarEdad(25) == "Adultez"	edad < 60
TC07	Vejez	62	"Vejez"	clasificarEdad(60) == "Vejez"	else final

4. Añade además casos límite especialmente relevantes.

Están añadidos los valores normales y los límites en el apartado 1

5. Implementa ClasificarEdadTest.

implementada la clase metodoClasificarEdadTest en JUnit, he hecho una clase main llamada metodoClasificarEdad, tiene el método abajo y en el main he hecho una pequeña prueba que pide el numero y te da el resultado del método aparte de la clase metodoClasificarEdadTest en el apartado Test.

Ejercicio 3. Método contarPositivos

Se proporciona el siguiente método:

```
 public static int contarPositivos(int[] datos) {
 int contador = 0;
 for (int i = 0; i < datos.length; i++) {
 if (datos[i] > 0) {
 contador++;
        }
   }
return contador;
}
```

Tareas

1. Análisis del método o programa

• Qué hace.

Cuenta cuántos números positivos hay en un array.

• Qué entradas admite.

Es un array de números enteros, permite negativos, positivos y 0 , o puede estar vacio

• Qué salidas produce.

La cantidad de números positivos que haya en el array

• Casos normales y casos límite.

Casos normales

[1, 2, 3] – 3

[-1, -2] – 0

[1, -2, 0, 3] – 2

Casos límite

[] – array vacío – resultado 0

[0, 0, 0] – la cantidad que fuese de 0 no contaría como positivo

todos negativos – resultado 0

2. Calcula laDecisiones:

for → 1

if (datos[i] > 0) → 1

Tenemos dos decisiones por tanto

M = 2 + 1 = 3 de complejidad ciclomática.

Hay que hacer mínimo 3 tests.

3. Diseña pruebas para cubrir:
• array vacío,
• array con todos negativos,
• array con mezcla de positivos, negativos y ceros.

ID	Descripción	Entrada	Esperado	Oráculo	Camino
TC01	Array vacío	{}	0	contarPositivos({}) == 0	No entra en el for
TC02	Todos negativos	{-1, -2, -3}	0	contarPositivos(...) == 0	Entra en for pero no en if
TC03	Mezcla de valores	{1, -2, 0, 3}	2	contarPositivos(...) == 2	Entra en for y en if algunas veces

4. Justifica qué ramas cubre cada prueba.

TC01 → cubre caso donde el for NO se ejecuta
TC02 → cubre ejecución del for pero NO del if
TC03 → cubre ejecución de ambos (for + if)

5. Implementa ContarPositivosTest.

En este no he hecho main, he creado la clase metodoContarPositivos, le he añadido el metodo al final y luego he creado la clase metodoContarPositivosTest en su respectivo sitio, y he implementado los test y comprobado que iban.

Ejercicio 4. Método calificación

Se proporciona el siguiente método:

```
 public static String calificacion(int nota) {
 if (nota < 0 || nota > 10) {
throw new IllegalArgumentException("Nota no válida");
 }
return switch (nota) {
case 0, 1, 2, 3, 4 -> "SUSPENSO";
 case 5 -> "SUFICIENTE";
 case 6 -> "BIEN";
case 7, 8 -> "NOTABLE";
 default -> "SOBRESALIENTE";
          };
 }
```

Tareas

1. Análisis del método o programa

• Qué hace.

Según el número que introduzcas dará una categoría de nota

• Qué entradas admite.

Un número entero, k sera la nota

• Qué salidas produce.

"SUSPENSO"
"SUFICIENTE"
"BIEN"
"NOTABLE"
"SOBRESALIENTE"
- Excepción si la nota fuese < 0 o > 10

- **Casos normales y casos límite.**

Casos normales

3 – SUSPENSO
5 – SUFICIENTE
6 – BIEN
7 – NOTABLE
9 – SOBRESALIENTE

Casos límite

-1 – excepción
0 – SUSPENSO
4 – SUSPENSO
5 – SUFICIENTE
6 – BIEN
7 – NOTABLE
8 – NOTABLE
9 – SOBRESALIENTE
10 – SOBRESALIENTE
11 – excepción

2. Calcula la complejidad ciclomática

Tenemos 6 caminos , el de la excepcion si es menos que 0 o mayor que 10 y las 5 notas

M=6+1

Hay que hacer mínimo 7 tests.

3. Diseña los casos mínimos para cubrir todas las alternativas del switch y la validación inicial.

ID	Descripción	Entrada	Esperado	Oráculo	Camino
TC01	Nota inválida baja	-1	Excepción	lanza IllegalArgumentException	validación
TC02	Nota inválida alta	11	Excepción	lanza IllegalArgumentException	validación
TC03	Suspenso	3	"SUSPENSO"	calificacion(3) == "SUSPENSO"	case 0–4
TC04	Suficiente	5	"SUFICIENTE"	calificacion(5) == "SUFICIENTE"	case 5
TC05	Bien	6	"BIEN"	calificacion(6) == "BIEN"	case 6
TC06	Notable	7	"NOTABLE"	calificacion(7) == "NOTABLE"	case 7–8
TC07	Sobresaliente	9	"SOBRESALIENTE"	calificacion(9) == "SOBRESALIENTE"	default

4. Implementa CalificacionTest.

Clase metodoCalificacion para meter el método y metodoCalificacionTest implementados en sus respectivos lugares y test probados

5. Indica qué casos podrían agruparse con @ParameterizedTest.

podríamos agrupar todos los suspensos (0–4)
y todos los notables (7–8)

Ejercicio 5. **Crear un método esBisiesto**

Implementa un método con la siguiente firma:

public static boolean esBisiesto(int anyo)

Reglas
Un año es bisiesto si:
• es divisible por 4 y no por 100, o
• es divisible por 400.
Tareas
1. Análisis del método o programa
• Qué hace.
Nos dice si el año que introducimos es bisiesto
• Qué entradas admite.
número esteros
• Qué salidas produce.
true o false depende de si el año que le pasemos es o no bisiesto
• Casos normales y casos límite.

2. Implementa el método.
Está implementado en la clase metodoEsBisiesto

```
public static boolean esBisiesto(int anyo) {
        if ((anyo % 4 == 0 && anyo % 100 != 0) || (anyo % 400 == 0))
{
            return true;
        }else {
        return false;}
    }
```

3. Calcula su complejidad ciclomática a partir de tu código.

M = 3 + 1 = 4

Hay que hacer mínimo 4 tests.

4. Diseña los casos de prueba necesarios para cubrir todas las decisiones.

ID	Descripción	Entrada	Esperado	Oráculo	Camino
TC01	Divisible por 4 y no por 100	2024	true	esBisiesto(2024) == true	primera condición
TC02	Divisible por 100 y no por 400	1900	false	esBisiesto(1900) == false	caso especial (rechazo)
TC03	Divisible por 400	2000	true	esBisiesto(2000) == true	excepción válida
TC04	No divisible por 4	2023	false	esBisiesto(2023) == false	caso negativo general

5. Implementa EsBisiestoTest. Requisitos Debes incluir al menos estos tipos de caso:
• divisible por 4 y no por 100,
• divisible por 100 y no por 400,
• divisible por 400.

Esta implementado en su respectivo apartado

Ejercicio 6. Implementar Piedra – Papel – Tijera

Implementa el método:

public static String jugar(String jugador1, String jugador2)

Las jugadas posibles son:

- **"PIEDRA"**
- **"PAPEL"**
- **"TIJERA"**

Reglas

- **Piedra gana a tijera**
- **Tijera gana a papel**
- **Papel gana a piedra**
- **Si ambas jugadas son iguales --> "EMPATE"**
- **Si alguna jugada no es válida --> IllegalArgumentException**

Tareas

1. Análisis del método o programa

- **Qué hace.**

Simula el juego Piedra–Papel–Tijera entre dos jugadores.

- **Qué entradas admite.**

Admitirá 2 strings entre
"PIEDRA"
"PAPEL" o
"TIJERA".

- **Qué salidas produce.**

"JUGADOR1"
"JUGADOR2"
"EMPATE" o
Excepción si la jugada es inválida

- **Casos normales y casos límite.**

2. Implementar el método.

3. Calcular la complejidad ciclomática.

M = 5 + 1 = 6

Hay que hacer mínimo 6 tests

4. Diseñar los tests mínimos para cubrir todas las ramas.

ID	Descripción	Entrada	Esperado	Oráculo	Camino
TC01	Jugada inválida	("PIEDRA", "FUEGO")	Excepción	lanza IllegalArgumentException	validación
TC02	Empate	("PIEDRA", "PIEDRA")	EMPATE	jugar(...) == "EMPATE"	empate
TC03	Gana jugador1 (piedra)	("PIEDRA", "TIJERA")	JUGADOR1	jugar(...) == "JUGADOR1"	condición jugador1
TC04	Gana jugador1 (tijera)	("TIJERA", "PAPEL")	JUGADOR1	jugar(...) == "JUGADOR1"	condición jugador1
TC05	Gana jugador1 (papel)	("PAPEL", "PIEDRA")	JUGADOR1	jugar(...) == "JUGADOR1"	condición jugador1
TC06	Gana jugador2	("PIEDRA", "PAPEL")	JUGADOR2	jugar(...) == "JUGADOR2"	resto de casos

5. Implementar PiedraPapelTijeraTest.
Implementada la clase metodoPiedraPapelTijera con el método PiedraPapelTijera y su respectiva clase metodoPiedraPapelTijeraTest.

Ejercicio 7. **Implementar Validador de contraseñas**

Implementa el método:

public static int evaluarPassword(String password)

Reglas

Se asignan puntos:

- **+1 si tiene al menos 8 caracteres**
- **+1 si tiene mayúsculas**
- **+1 si tiene minúsculas**
- **+1 si tiene números**
- **+1 si tiene símbolos**

Interpretación del resultado:

- **entre 0 y 2 --> "DEBIL"**
- **entre 3 y 4 --> "MEDIA"**
- **5 --> "FUERTE"**

Tareas

1. Análisis del método o programa

- **Qué hace.**

Puntua la seguridad de una contraseña basandose en los criterios k le hemos puesto

- **Qué entradas admite.**

admite un String , que seria la contraseña que queramos poner

- **Qué salidas produce.**

Usará un entero entre 1 y 5 incluidos y luego interpretaremos nosotros el resultado segun la puntuacion de la contraseña que sera el numero que ha salido

- **Casos normales y casos límite.**

Limite

Que la contraseña tenga solo caracteres, solo numeros,solo letras o un batiburrillo de todas

2. Implementar el método.

```
public static int evaluarPassword(String password) {
    int puntos = 0;

    if (password.length() >= 8) puntos++;

    if (password.matches(".*[A-Z].*")) puntos++;

    if (password.matches(".*[a-z].*")) puntos++;

    if (password.matches(".*[0-9].*")) puntos++;

    if (password.matches(".*[^a-zA-Z0-9].*"))
puntos++;

    return puntos;
}
```

3. Calcular complejidad ciclomática.

Hay 5 if , 5 decisiones y se le suma 1 y tenemos la M

M=5+1

4. Diseñar casos de prueba representativos.

ID	Descripción	Entrada	Esperado	Oráculo	Camino
TC 01	Muy corta	"abc"	1	evaluarPassword ("abc") == 1	longitud corta + min
TC 02	Solo minúsculas	"abcdefgh"	2	evaluarPassword (...) == 2	min + longitud
TC 03	Minúsculas + números	"abc12345"	3	evaluarPassword (...) == 3	+ números
TC 04	Añade mayúscula a lo otro	"Abc12345"	4	evaluarPassword (...) == 4	+ mayúscula
TC 05	Completa	"Abc123$%"	5	evaluarPassword (...) == 5	todos
TC 06	Solo números	"12345678"	2	evaluarPassword (...) == 2	longitud + num

5. Implementar PasswordTest.

Implementado en su correspondiente apartado y testeado.